you declare that you want three replicas of your Spring Boot application deployed. If one crashes, the associated ReplicaSet notices it and creates a new Pod to align the actual state with the desired one.

After packaging a Spring Boot application as an OCI image, all you need to do to run it in a Kubernetes cluster is define a Deployment object. You'll learn how in the next section.

7.2.3 Creating a Deployment for a Spring Boot application

There are a few options for creating and managing Kubernetes objects in a cluster. In chapter 2 we used the kubectl client directly, but that approach lacks version control and reproducibility. It's the same reason why we prefer Docker Compose over the Docker CLI.

In Kubernetes, the recommended approach is to describe an object's desired state in a *manifest* file, typically specified in YAML format. We use *declarative configuration*: we declare what we want instead of how to achieve it. In chapter 2 we *imperatively* used kubectl to create and delete objects, but when we work with manifests, we *apply* them to the cluster. Then Kubernetes will automatically reconcile the actual state in the cluster with the desired state in the manifest.

A Kubernetes manifest usually comprises four main sections, as shown in figure 7.5: